

they click a submission button, but for a model to run as a user is editing text, it would need to run significantly under a second. The most powerful models take longer to process data, so as we iterate through models, we will keep this requirement in mind. Any model we use should be able to process an example through its whole pipeline in under two seconds.

Model inference runtimes increase as models get more complex. The difference is significant even in a domain where each individual data point can be relatively small data such as NLP (as opposed to tasks on live videos, for example). On the text data used for the case study in this book, for example, an LSTM is about three times slower than a random forest (around 22 ms for an LSTM, while the random forest takes only 7 ms). On an individual datapoint, this difference are small, but they can quickly add up when needing to run inference on tens of thousands of examples at a time.

For complex applications where an inference call will be associated with multiple network calls or database queries, model execution time can become short compared to the rest of the application logic. In those cases, the speed of said models becomes less of an issue.

Depending on your problem, there are other categories you could consider, such as hardware constraints, development time, and maintainability. It is important to develop an understanding of your needs before choosing a model so that you make sure to pick said model in an informed way.

Once you identify requirements and associated metrics, it's time to start making a plan. This requires estimating the challenges that lie ahead. In the next section, I'll cover ways to leverage prior work and explore a dataset to decide what to build next.

Estimate Scope and Challenges

As we've seen, ML performance is often reported in terms of model metrics. While these metrics are useful, they should be used to improve the product metrics we defined, which represent the actual task we are trying to solve. As we iterate on a pipeline, we should keep in mind product metrics and aim to improve them.

The tools we have covered so far will help us determine whether a project is worth tackling at all, and measure how well we are currently doing. A logical next step is to sketch out a plan of attack to estimate the scope and duration of a project and anticipate potential roadblocks.

In ML, success generally requires understanding the context of the task well, acquiring a good dataset, and building an appropriate model.

We will cover each of these categories in the following section.

Leverage Domain Expertise

The simplest model we can start with is a heuristic: a good rule of thumb based on knowledge of the problem and the data. The best way to devise heuristics is to see what experts are currently doing. Most practical applications are not entirely novel. How do people currently solve the problem you are trying to solve?

The second best way to devise heuristics is to look at your data. Based on your dataset, how would you solve this task if you were doing it manually?

To identify good heuristics, I recommend either learning from experts in the field or getting familiar with the data. I'll describe both in a little more detail next.

Learning from experts

For many domains we might want to automate, learning from experts in the domain can save us dozens of hours of work. If we are attempting to build a predictive maintenance system for factory equipment, for example, we should start by reaching out to a factory manager to understand which assumptions we can reasonably make. This could include understanding how often maintenance is currently performed, which symptoms usually indicate that a machine will require maintenance soon, and the legal requirements concerning maintenance.

There are, of course, examples where finding domain experts is likely to be difficult—such as proprietary data for a novel use case like predicting usage of a unique website feature. In these cases, however, we can often find professionals who have had to tackle similar problems and learn from their experiences.

This will allow us to learn about useful features we can leverage, find pitfalls we